

Mezclando C y assembler en AVR

El entorno de programación AVR Studio permite mezclar los lenguajes de programación C y assembler para realizar programas para los microcontroladores. Para llevar esto adelante se deben realizar los siguientes pasos.

Pasando una rutina de assembler a C:

En el archivo de C, se deben declarar las funciones tipo de las rutinas a traer desde assembler, en este caso será la rutina “delay_joel”.

```
extern void delay_joel ();
```

El archivo con el código assembler:

Se debe crear un archivo “.S” en el cual se escribe el código assembler de las rutinas. En el mismo se deben hacer las siguientes declaraciones al comienzo y al final.

```
.global delay_joel;  
.section .text
```

Aquí Código

Ret

Agregando el archivo “.S” al proyecto:

Por defecto el AVR studio no reconoce el archivo “.S” generado y debemos agregarlo al proyecto para esto hacemos lo que se muestra en la Fig.1.

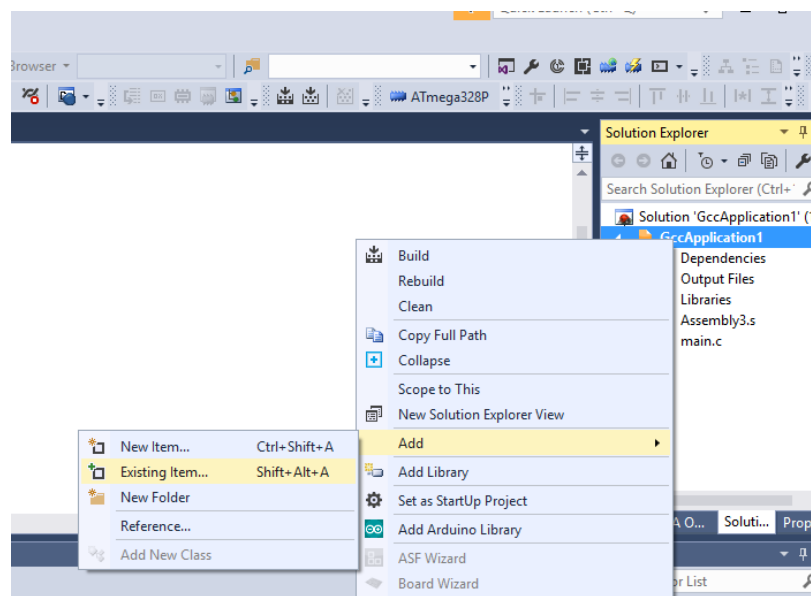


Figura 1. Agregando un archivo al proyecto.

Se busca el archivo “.S” y se da aceptar. Con esto quedaría agregado al proyecto.

Pasando datos a los registros:

En primer lugar, es importante saber que al utilizar C como lenguaje principal para el desarrollo de programas para el microcontrolador, no todos los registros disponibles son de libre uso. En la Tabla 1 se muestran cuales están restringidos y cuáles son libres. Los restringidos son así debido a que el compilador los va a utilizar para generar el código assembler a partir del compilado de C.

Register	Description	Assembly code called from C	Assembly code that calls C code
r0	Temporary	Save and restore if using	Save and restore if using
r1	Always zero	Must clear before returning	Must clear before calling
r2-r17	"call-saved"	Save and restore if using	Can freely use
r28			
r29			
r18-r27	"call-used"	Can freely use	Save and restore if using
r0			
r31			

Tabla 1. Características de los registros.

En caso de utilizar los registros que no son libres, deben ser guardados antes de llamar a nuestra rutina mediante el comando **push** y luego restaurados mediante **pop**.

Para acceder a un registro desde C lo vemos como una dirección de memoria. En la Fig. 2 se muestra la memoria del microcontrolador.

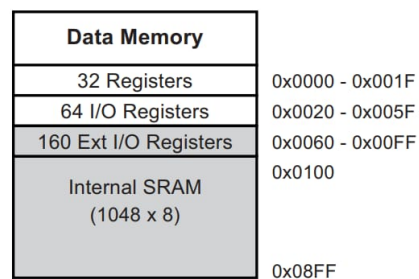


Figura 2. Memoria de micocotrolador ATmega328p.

De la Figura 2 se puede determinar por ejemplo que el registro r18 es la dirección 0x12. Para poder escribir en dicho registro debemos hacer un apuntador a esa dirección de memorias, a continuación se muestra un ejemplo para los registros de del r18-r20

```
unsigned char *pr18 = 0x12;  
unsigned char *pr19 = 0x13;  
unsigned char *pr20 = 0x14;
```

Para escribir un valor en los registros apuntados se debe hacer lo siguiente.

```
*pr18=0xFF; // cargo valor 0xFF en registro r18  
*pr19=0xFF; // cargo valor 0xFF en registro r19  
*pr20=0x20; // cargo valor 0x20 en registro r20
```

Se deja disponible en la webasignatura el ejemplo completo .